



**Instituto Tecnológico y de Estudios Superiores de Monterrey**

Campus Estado de México, Escuela de Ingeniería y Ciencias.

**Unidad de formación:**

TC3009C.601

**Inteligencia artificial avanzada para la ciencia de datos.**

**Momento de Retroalimentación: Módulo 2 Implementación de una técnica de aprendizaje máquina sin el uso de un framework. (Portafolio Implementación)**

Aarón Ramírez Pulido

A01749666

02 de septiembre de 2026

Profesor: Jorge Adolfo Ramírez Uresti

## Introducción y objetivo

El objetivo de esta actividad fue implementar manualmente un algoritmo de Decision Tree para un problema de clasificación multiclase, sin utilizar frameworks de Machine Learning que proporcionaran el algoritmo ya implementado. El modelo se desarrolló en Python y se probó con el UCI Wine Dataset.

La implementación incluye de forma manual: Gini impurity, weighted Gini, búsqueda del mejor feature y threshold, construcción recursiva del árbol, predicción, matriz de confusión y métricas de clasificación. El archivo final se ejecuta directamente desde consola mediante `python decision_tree.py`, sin depender de un notebook o IDE.

## Dataset utilizado

Se utilizó el Wine Dataset del UCI Machine Learning Repository. El problema consiste en clasificar cada muestra de vino en una de tres clases a partir de sus características químicas.

| Característica     | Valor |
|--------------------|-------|
| Observaciones      | 178   |
| Features numéricos | 13    |
| Clases             | 3     |
| Valores faltantes  | 0     |

Los 13 features son: Alcohol, Malic acid, Ash, Alcalinity of ash, Magnesium, Total phenols, Flavanoids, Nonflavanoid phenols, Proanthocyanins, Color intensity, Hue, OD280/OD315 of diluted wines y Proline.

**Fuente:** [UCI Machine Learning Repository - Wine Dataset](#)

## Implementación sin framework

El código utiliza únicamente módulos estándar de Python: `csv`, `random`, `pathlib.Path`. No se utilizaron Scikit-learn, TensorFlow, Keras ni librerías de estadística avanzada.

El Decision Tree selecciona en cada nodo la división con menor Gini ponderado. El Gini impurity se calcula como  $Gini = 1 - \sum p_i^2$ , donde  $p_i$  es la proporción de observaciones de cada clase. Para cada feature se prueban thresholds ubicados entre valores consecutivos y se conserva la división que produce la menor impurity.

| Función                                | Propósito                                      |
|----------------------------------------|------------------------------------------------|
| <code>load_wine_data()</code>          | Carga el dataset y separa features y clase.    |
| <code>gini_impurity()</code>           | Calcula manualmente Gini impurity.             |
| <code>find_best_split()</code>         | Busca el mejor feature y threshold.            |
| <code>build_tree()</code>              | Construye recursivamente nodos y hojas.        |
| <code>predict()</code>                 | Genera predicciones para nuevas observaciones. |
| <code>create_confusion_matrix()</code> | Construye manualmente la matriz de confusión.  |
| <code>calculate_class_metrics()</code> | Calcula Precision, Recall y F1-score.          |

## Entrenamiento, validación y prueba

Para evitar evaluar el modelo con los mismos datos utilizados para construirlo, el dataset se dividió con una semilla fija de 42 en tres subconjuntos:

- Training set: 126 observaciones (70.8%).
- Validation set: 26 observaciones (14.6%).
- Test set: 26 observaciones (14.6%).

El training set se utilizó para construir el árbol; el validation set para seleccionar la profundidad máxima; y el test set se reservó para la evaluación final.

## Selección de profundidad

| Max depth | Validation Accuracy |
|-----------|---------------------|
| 1         | 57.69%              |
| 2         | 80.77%              |
| 3         | 84.62%              |
| 4         | 84.62%              |
| 5         | 84.62%              |
| 6         | 84.62%              |

Las profundidades 3 a 6 obtuvieron el mismo Validation Accuracy de 84.62%. Se seleccionó `max_depth = 3` porque logra el mejor desempeño con la menor complejidad entre las alternativas empatadas.

```
PS C:\Users\aaaron\Downloads\machine-learning-sin-framework-main\machine-learning-sin-framework-main>
UCI Wine Dataset
-----
Number of observations: 178
Number of features: 13
Classes found: [1, 2, 3]

Dataset Split
-----
Training observations: 126
Validation observations: 26
Testing observations: 26

Class Distribution
-----
Training: {3: 33, 2: 49, 1: 44}
Validation: {3: 8, 2: 14, 1: 4}
Testing: {3: 7, 1: 11, 2: 8}

Model Selection
-----
Depth 1: 0.5769 (57.69%)
Depth 2: 0.8077 (80.77%)
Depth 3: 0.8462 (84.62%)
Depth 4: 0.8462 (84.62%)
Depth 5: 0.8462 (84.62%)
Depth 6: 0.8462 (84.62%)

Selected depth: 3
Best validation accuracy: 0.8462 (84.62%)

Building Final Decision Tree
-----
```

Figura 1. Selección de profundidad y construcción del modelo desde consola.

## Resultados

El modelo final se evaluó únicamente con las 26 observaciones del test set. Clasificó correctamente 23 de 26 observaciones, obteniendo un Accuracy de 88.46%.

| Real / Pred. | Clase 1 | Clase 2 | Clase 3 |
|--------------|---------|---------|---------|
| Clase 1      | 11      | 0       | 0       |
| Clase 2      | 1       | 6       | 1       |
| Clase 3      | 0       | 1       | 6       |

## Métricas de clasificación

| Clase         | Precision | Recall | F1-score |
|---------------|-----------|--------|----------|
| Clase 1       | 0.9167    | 1.0000 | 0.9565   |
| Clase 2       | 0.8571    | 0.7500 | 0.8000   |
| Clase 3       | 0.8571    | 0.8571 | 0.8571   |
| Macro average | 0.8770    | 0.8690 | 0.8712   |

La primera decisión encontrada por el árbol fue  $\text{Proline} \leq 755.0$ , con weighted Gini de 0.3779. Esto muestra que Proline fue el feature más útil para la primera separación del training set.

```

First 10 Test Predictions
-----
Wine 1: Real class = 3 | Predicted class = 3
Wine 2: Real class = 1 | Predicted class = 1
Wine 3: Real class = 1 | Predicted class = 1
Wine 4: Real class = 2 | Predicted class = 2
Wine 5: Real class = 1 | Predicted class = 1
Wine 6: Real class = 3 | Predicted class = 2
Wine 7: Real class = 2 | Predicted class = 2
Wine 8: Real class = 2 | Predicted class = 2
Wine 9: Real class = 2 | Predicted class = 2
Wine 10: Real class = 1 | Predicted class = 1

Confusion Matrix
-----
Actual \ Predicted    1     2     3
Class 1              11     0     0
Class 2               1     6     1
Class 3               0     1     6

Classification Metrics
-----
Accuracy: 0.8846 (88.46%)

Class 1:
Precision: 0.9167
Recall:    1.0000
F1-score:  0.9565

Class 2:
Precision: 0.8571
Recall:    0.7500
F1-score:  0.8000

Class 3:
Precision: 0.8571
Recall:    0.8571
F1-score:  0.8571

Macro Averages
-----
Precision: 0.8770
Recall:    0.8690

```

Figura 2. Matriz de confusión y métricas obtenidas en el test set.

## Análisis y conclusión

Los resultados muestran que la implementación manual del Decision Tree funciona correctamente para este problema. La Clase 1 presentó el mejor desempeño: las 11 observaciones reales de esa clase fueron identificadas correctamente, con Recall = 1.0000. La Clase 2 fue la más difícil de clasificar, con Recall = 0.7500; de ocho observaciones, seis fueron correctas, una se confundió con Clase 1 y una con Clase 3. Para la Clase 3 se clasificaron correctamente seis de siete observaciones.

El Accuracy final de 88.46% y el macro F1-score de 0.8712 indican un desempeño adecuado y relativamente equilibrado entre las tres clases. Además, el uso de un validation set permitió seleccionar la profundidad del árbol sin utilizar el test set para ajustar el modelo.

Como limitación, el dataset es pequeño y el test set contiene únicamente 26 observaciones, por lo que el 88.46% corresponde al desempeño en esta partición específica y no debe interpretarse como una estimación universal. Aun así, el proyecto cumple el objetivo de demostrar que un Decision Tree puede implementarse desde cero, ejecutarse de forma independiente y generar predicciones y métricas consistentes sin recurrir a un framework de Machine Learning.

## Repositorio y referencias

Repositorio de GitHub: [github.com/apachai-aaron/machine-learning-sin-framework](https://github.com/apachai-aaron/machine-learning-sin-framework)

Referencia del dataset: UCI Machine Learning Repository. (s. f.). Wine. University of California, Irvine. <https://archive.ics.uci.edu/dataset/109/wine>

Archivos principales del repositorio: `decision_tree.py`, `wine.data`, `wine.names` y `README.md`.